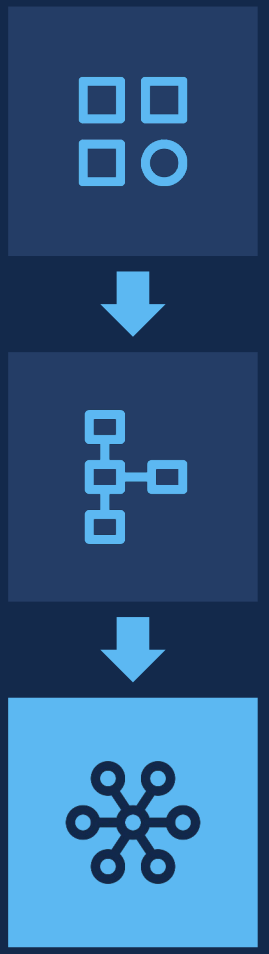
경남대학교 피지컬AI 사관학교

설비와 데이터의 관계를 체계화해 필요한 정보 찾기

지식 구조의 설계와 그래프 조회

현장 정보를 클래스와 관계로 나누어 스키마로 정의하고, 그래프에 저장해 관계를 따라 찾는다



UEngine

학습목표

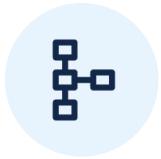
현장 정보를 클래스와 관계로 나누어 스키마로 정의하고, 그래프 DB에 저장해 관계를 따라 조회한다.



클래스 · 인스턴스와 관계

- 현장의 대상을 클래스, 인스턴스, 속성값으로 나눈다
- 관계의 방향과 카디널리티를 실제 연결에 대어 읽는다

분류와 표현



스키마 설계

- 클래스 · 속성 · 관계 · 제약을 한 벌의 정의로 적는다
- 식별자로 같은 대상을 가리키고 어긋난 데이터를 가려낸다

정의와 제약



그래프 DB와 Cypher 쿼리

- 노드 · 관계 · 속성의 저장 구조를 이해한다
- Cypher로 조회 · 집계하고 노드와 관계를 만든다

저장 · 조회 · 생성



설계와 구현의 검증

- 설비 자료에서 스키마와 예시 인스턴스를 설계한다
- 그래프 DB에 넣고 조회 결과를 설계와 대조한다

결과물 검증

목차

01	클래스 · 인스턴스와 관계의 기초	이론
02	카디널리티와 관계의 속성	이론
03	클래스 계층과 모델링 선택	이론
04	스키마와 식별자	이론
05	속성과 연결의 정의 · 제약	이론
06	설비 모델의 설계와 확장	이론
07	그래프 DB의 저장 구조와 Cypher 쿼리 조회	이론
08	Cypher 쿼리: 노드 · 관계 생성과 병합	이론
09	관계 조회 · 기준 지식 탐색 · 온톨로지 설계 · 그래프 구현	실습

01 클래스 · 인스턴스와 관계의 기초

이론

유온 센서의 이름과 값을 적고, 측정 대상과 주변 장치를 관계로 잇는다

클래스와 인스턴스

클래스는 공통 특성을 가진 대상의 유형이고, 인스턴스는 그 유형에 속하는 개별 대상이다.

클래스(Class)

- 공통 특성을 가진 대상의 유형
- 설비(Asset), 센서(Sensor)
- 같은 유형이 사용할 속성과 관계를 정한다

공통 정의

vs

인스턴스(Instance)

- 클래스에 속하는 개별 대상
- Asset: HYD-01 · HYD-02 · HYD-03
- Sensor: 유온 센서 · 냉각 효율 채널
- 「유온 센서는 센서의 하나이다」

개별 대상

센서 클래스의 속성과 속성값

속성(Property)은 값을 담는 항목이고, 그 항목에 채운 내용은 속성값(Property Value)이다.

속성	유온 센서	냉각 효율 채널
식별자(id)	sen:ts1	**sen:ce**
이름(name)	유온 센서	**냉각 효율**
태그(tag)	TS1	**CE**
단위(unit)	℃	**%**
계산 채널 여부(virtual)	false: 직접 계측	**true: 다른 계측값으로 계산**

센서와 상태 변수

한계를 센서가 아닌 상태 변수에 두어야, 센서가 바뀌거나 늘어도 비교할 기준이 하나로 남는다.

센서(Sensor)

- 값을 내놓는 계측 채널
- 유온 센서: sen:ts1
- 태그 TS1 · 단위 ℃
- 무엇으로 재는가

재는 쪽

vs

상태 변수(StateVariable)

- 설비의 상태를 나타내는 물리량
- 유온: sv:ts1
- 정상값(normal) 48 ℃ · 한계(limit) 65 ℃
- 무엇을 재고 어느 기준과 비교하는가

재어지는 쪽

데이터 속성과 객체 간 관계

태그는 센서의 속성값으로 적고, 정상값과 한계를 가진 유온은 별도 인스턴스로 연결한다.

데이터 속성(Data Property)

- 인스턴스에 값을 붙인다
- 유온 센서의 tag: 'TS1'
- 유온 센서의 unit: '℃'
- 유온의 limit: 65

인스턴스와 값

vs

객체 간 관계(Object Property)

- 인스턴스를 다른 인스턴스에 잇는다
- 유온 센서는 유온을 읽는다: OBSERVES
- 건너간 유온에서 한계(limit)를 읽는다

인스턴스와 인스턴스

설비 · 구성 요소 · 부품

설비 한 대, 설비를 이루는 장치, 교체할 부품을 구별하면 계측과 정비의 대상을 분명히 찾는다.

설비(Asset)
설비 한 대

설비 한 대를 나타낸다.
HYD-01 유압 파워팩 · asset:hyd-01

구성 요소(Component)
상태를 살피는 장치

설비를 이루는 장치이다.
오일 쿨러 · comp:cooler
유온 센서와 냉각 효율 채널로 상태를 확인한다.

부품(Part)
교체할 대상

구성 요소 안에서 갈아 끼우는 교체 부품이다.
쿨러 코어 · part:cooler-core
부품 번호: P-CLR-CORE

증상 · 고장 유형 · 원인

유온 상승은 관측된 증상, 냉각 성능 상실은 고장 유형, 쿨러 핀 오염은 그 원인에 해당한다.

증상(Symptom)
무엇이 보이는가

계측값에서 관측한 이상 양상이다.
유온 상승 · 냉각 효율 저하

고장 유형(FailureMode)
어떤 기능에 문제가 있는가

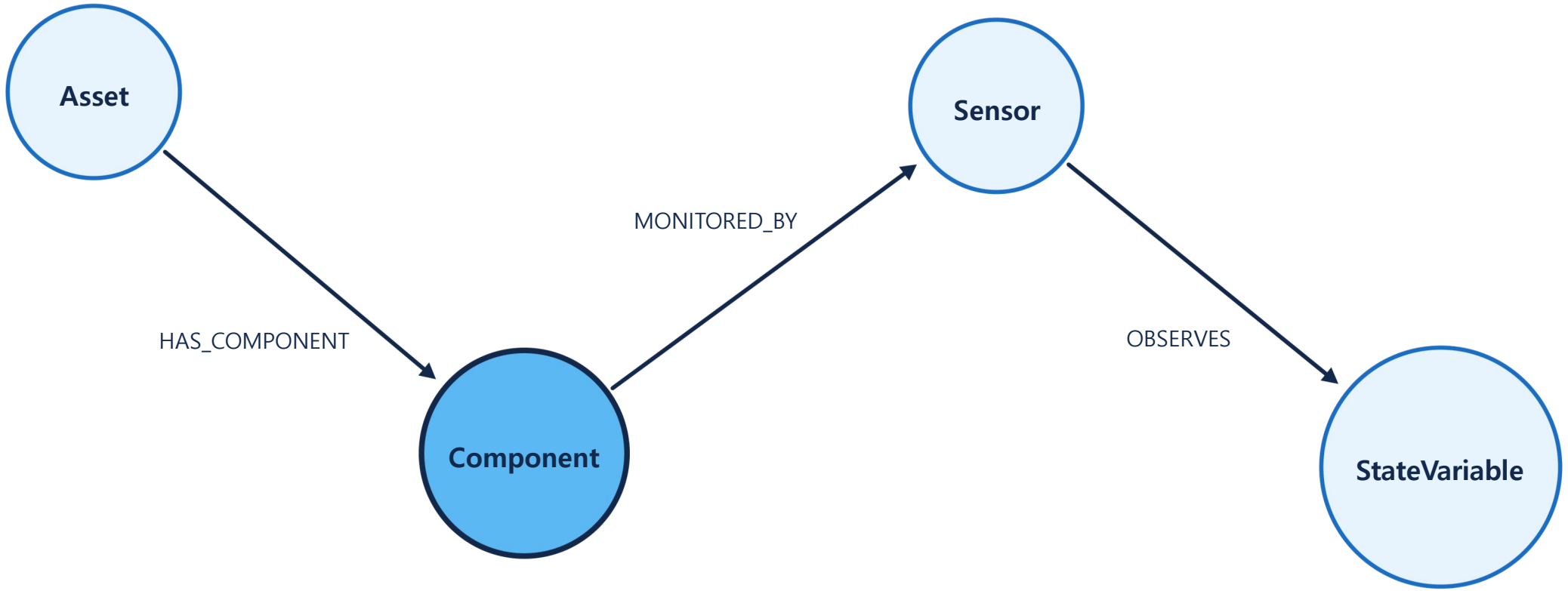
구성 요소에서 발생하는 고장의 양상이다.
쿨러 냉각 성능 상실

원인(Cause)
왜 문제가 생겼는가

고장을 일으키는 근본 원인이다.
쿨러 핀 오염 · 팬 구동 불량 · 주변 온도 상승

관계 유형의 이름과 방향

관계 유형(Relationship Type)은 연결의 이름과 방향, 시작 클래스와 끝 클래스를 정한다.



조작 관계와 물리적 영향의 구분

팬 인버터는 팬 속도를 바꾸고, 팬 속도가 유온을 낮추는 영향은 관계의 부호로 나타낸다.

조작 관계(MANIPULATES)

- 구동기(Actuator): PLC가 값을 써서 움직이는 장치
- 팬 인버터는 팬 속도를 바꾼다
- 시작: 팬 인버터 · 끝: 팬 속도

바꾸는 대상

vs

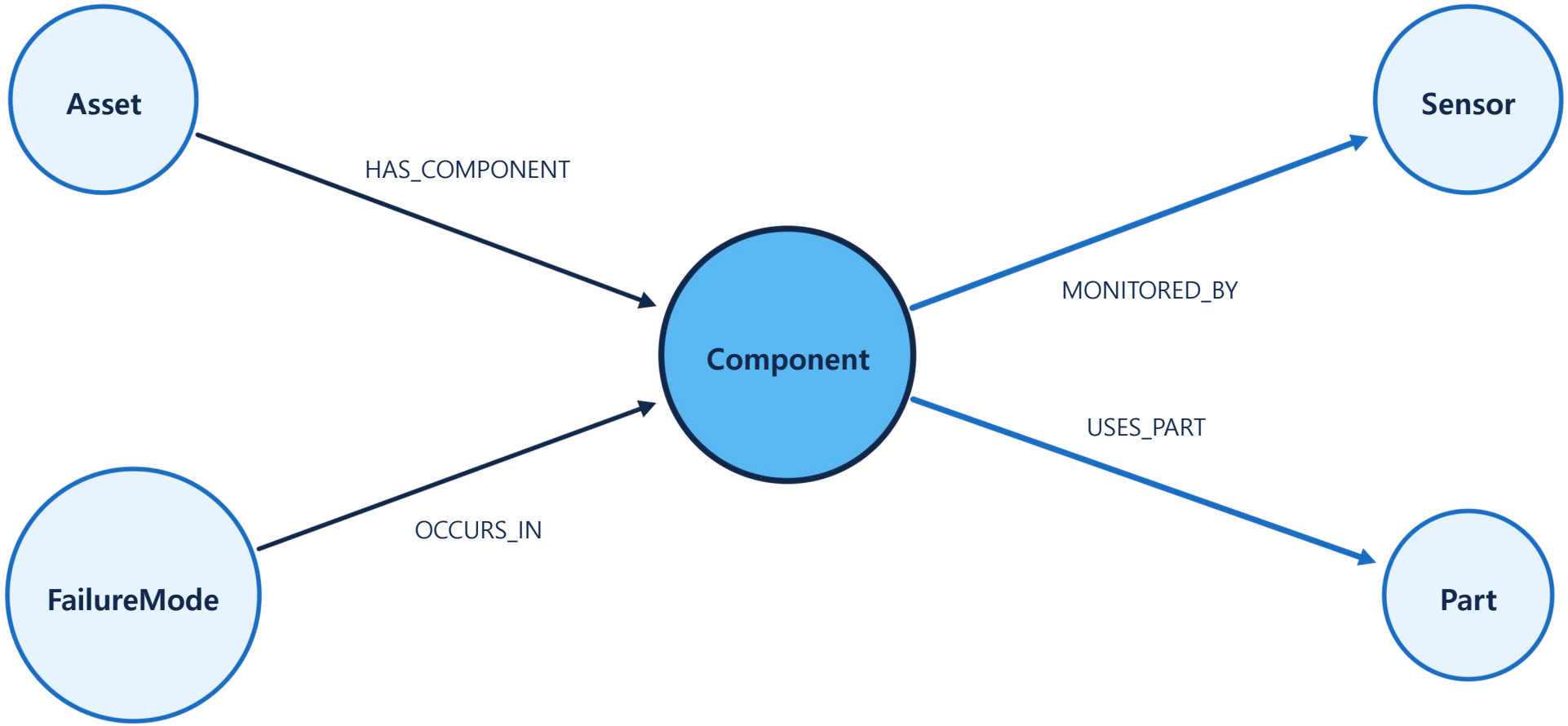
영향 관계(INFLUENCES)

- 팬 속도는 유온에 영향을 준다
- 관계의 속성(Relationship Property): 두 대상의 연결에 붙는 값
- sign: -1 → 팬 속도 증가, 유온 감소
- 견줄 것: 펌프 부하 → 유온은 sign +1

변화 방향

한 대상의 들어오는 관계와 나가는 관계

오일 쿨러의 설비와 고장은 들어오는 관계로, 계측 채널과 교체품은 나가는 관계로 찾는다.



02

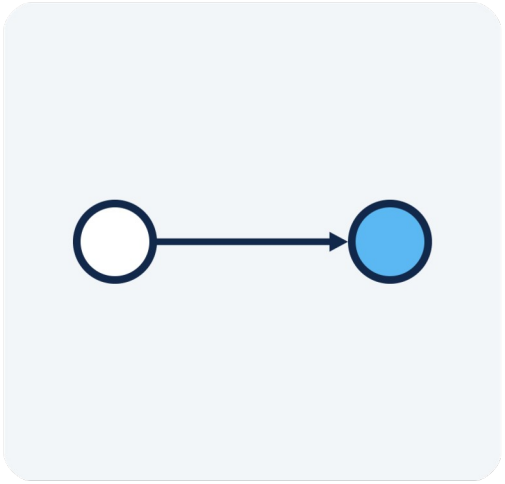
카디널리티와 관계의 속성

이론

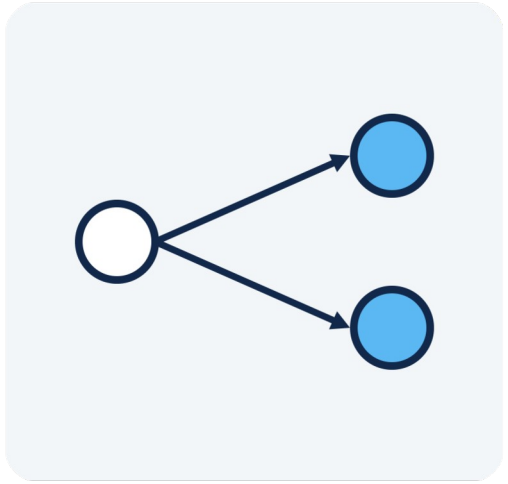
관계 양쪽의 연결 수를 읽고, 부품과 공급사의 짝에 붙는 값을 찾는다

카디널리티의 네 가지 형태

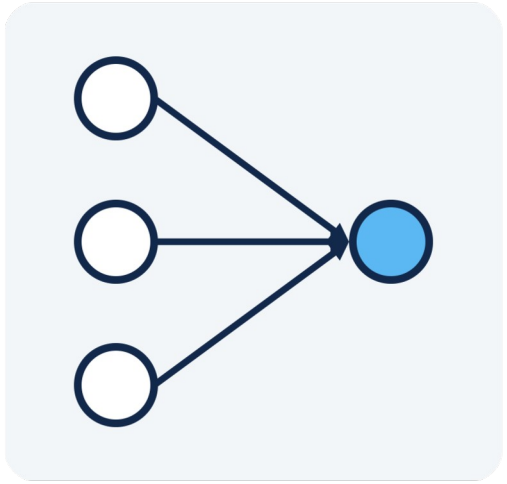
카디널리티(Cardinality)는 관계 한쪽의 대상 하나에 이어질 수 있는 반대쪽 대상의 수이다.



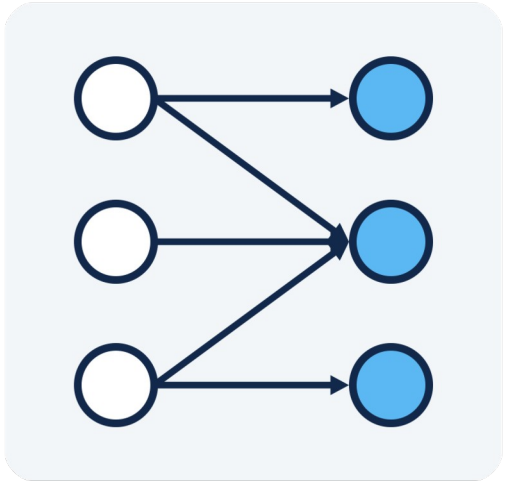
1:1 · MANIPULATES
팬 인버터 → 팬 속도



1:N · EVIDENCED_BY
쿨러 핀 오염 → 증거 둘



N:1 · OBSERVES
센서 여럿 → 상태 변수 하나



N:M · SUPPLIED_BY
부품 ↔ 공급사

카디널리티 선언과 실제 데이터

카디널리티는 스키마에 적은 선언이어서, 실제 데이터가 선언과 맞는지는 따로 대조해야 드러난다.

선언에 어긋난 데이터

- MANIPULATES는 1:1로 선언되어 있다
- 가정: 팬 인버터 → 팬 속도와 펌프 부하
- 구동기 하나에 상태 변수 둘이 이어졌다

다시 확인할 데이터

vs

선언과 다르게 쓰이는 관계

- HAS_COMPONENT는 1:N으로 선언되어 있다
- HYD-01 · HYD-02 · HYD-03이 같은 comp:cooler를 가리킨다
- 기종의 공통 구성 지식이며, 개별 쿨러를 구별하지 않는다
- 구성 요소 하나에 설비 셋이 이어져 다대다처럼 쓰인다

선언과 데이터의 차이

두 대상의 짝에 붙는 속성

부품과 공급사의 짝마다 달라지는 단가와 납기는 둘을 잇는 관계의 속성으로 둔다.

```
"type": "SUPPLIED_BY",
"from": ["Part"],  "to": ["Supplier"],
"cardinality": "N:M",
"properties": [
  {"name": "price",    "type": "number", "required": true},
  {"name": "failRate", "type": "number", "required": true},
  {"name": "leadDays", "type": "number"} ]

펌프 축 씰 키트의 공급 조건
A정밀:     price 35, failRate 0.12, leadDays 2
B-OEM:     price 55, failRate 0.02, leadDays 5
C트레이딩: price 20, failRate 0.30, leadDays 1
```

짝에 딸린 값

같은 부품도 공급사마다 값이 달라, 부품이나 공급사 한쪽에 하나로 둘 수 없다.

단위

price는 만원, leadDays는 일, failRate는 비율이다(0.12 = 12 %).

필수 여부

price와 failRate는 필수, leadDays는 선택이다.

03

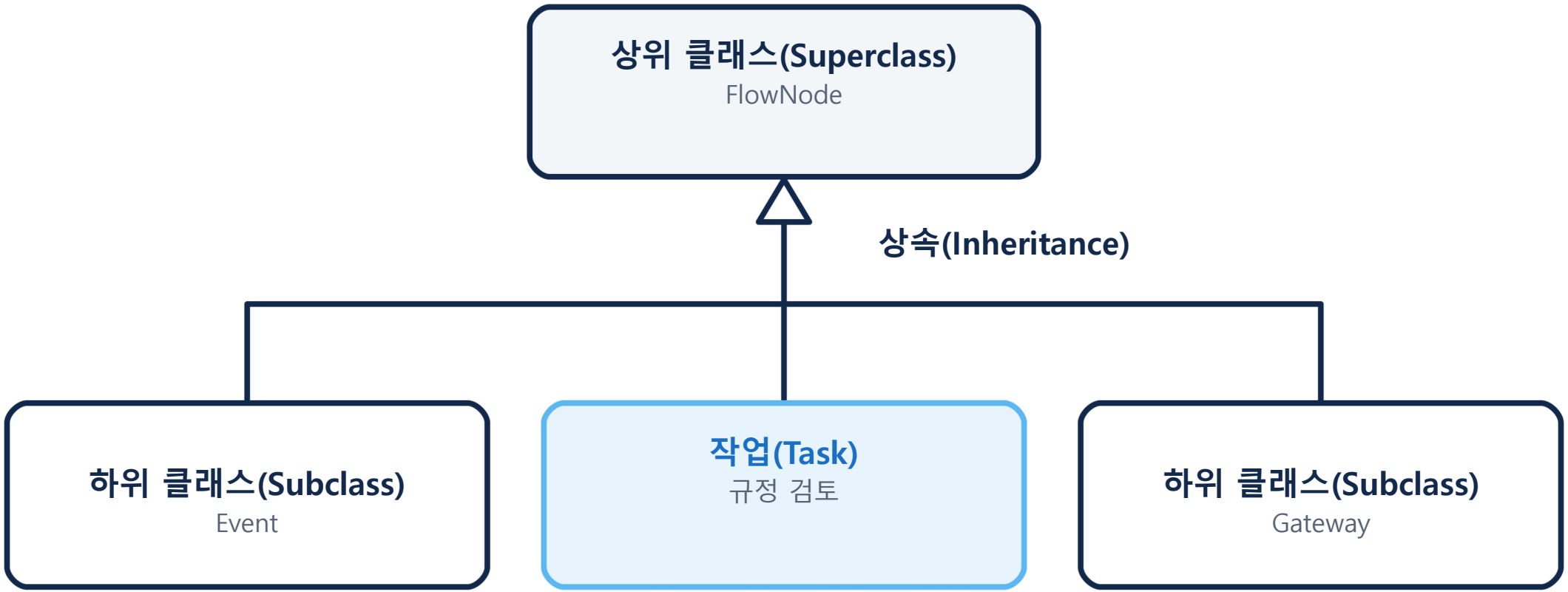
클래스 계층과 모델링 선택

이론

공통 정의를 물려받는 계층과, 새 클래스를 나눌 기준을 살펴본다

클래스 계층과 상속

FlowNode의 id·name을 세 하위 클래스가 **상속**하고, Task는 taskType을 더한다.



새 클래스를 정의하는 기준

갖는 속성과 맺는 관계가 다르면 클래스를 나누고, 같은 유형의 대상이 늘면 인스턴스를 더한다.

클래스를 나눈다

- 센서(Sensor): tag TS1 · unit ℃
- 상태 변수를 읽는다(OBSERVES)
- 구동기(Actuator): resource FanSpeedSP · min 0 · max 100
- 상태 변수를 바꾼다(MANIPULATES)

새 클래스

vs

인스턴스를 더한다

- 유온 센서 · 토출 압력 센서 · 팬 진동 센서는 속성과 관계가 같다
- 셋 모두 Sensor의 인스턴스로 둔다
- 이 모델의 질문 범위에서는 센서마다 클래스를 둘 까닭이 없다

기존 클래스

정보를 표현하는 네 가지 선택

정보 하나를 클래스 · 속성 · 관계 · 관계의 속성 가운데 어디에 둘지는 그 정보가 무엇에 딸렸는지로 정한다.

표현	고르는 기준과 실제 예	값이 딸린 자리
클래스	같은 속성과 관계를 갖는 대상이 여럿이다 예: Sensor, Actuator	유형
속성	한 대상에 딸린 값으로 끝난다 쿨러 코어: partNo P-CLR-CORE 공급사 A정밀: avl true	값
관계	자기 속성과 관계를 가진 다른 대상을 가리킨다 예: 오일 쿨러 → 쿨러 코어: USES_PART	연결
관계의 속성	두 대상의 짝마다 값이 달라진다 SUPPLIED_BY: price · leadDays 펌프 축 씰 키트 → A정밀: 35만원 · 2일	짝의 값

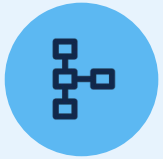
04 스키마와 식별자

이론

함께 쓸 정의를 정하고, 이름이 달라도 같은 대상을 찾는 기준을 둔다

스키마와 인스턴스 데이터

의미 불일치를 막으려고, **스키마**가 있을 수 있는 클래스 · 속성 · 관계를 먼저 정한다.



스키마(Schema)

- 클래스 · 속성 · 관계 · 제약의 정의만 담는다
- 실제 대상은 들어 있지 않다
- 예:「센서에는 tag가 있어야 한다」
- 넣는 사람과 읽는 프로그램이 같은 정의를 기준으로 삼는다

있을 수 있는 것

vs



인스턴스 데이터(Instance Data)

- 정의에 맞춰 적은 실제 대상과 그 사이의 관계이다
- 예:「유온 센서 sen:ts1의 tag는 TS1이다」
- 같은 기종 설비가 늘면 이쪽에 인스턴스를 더한다

실제로 있는 것

식별자 · 이름 · 별칭

같은 장치가 이름마다 다른 노드로 흩어지지 않게, 고유한 **식별자**로 가리키고 이름과 별칭은 따로 둔다.

식별자(Identifier)

- 대상 하나의 고유한 값이며 바꾸지 않는다
- 설비: asset:hyd-01 · asset:hyd-02 · asset:hyd-03
- 센서: sen:ts1
- 상태 변수: sv:ts1

id

이름(Name)

- 사람이 읽는 한국어 이름이며 겹치거나 바뀔 수 있다
- 「HYD-01 유압 파워팩」, 「오일 쿨러」

name

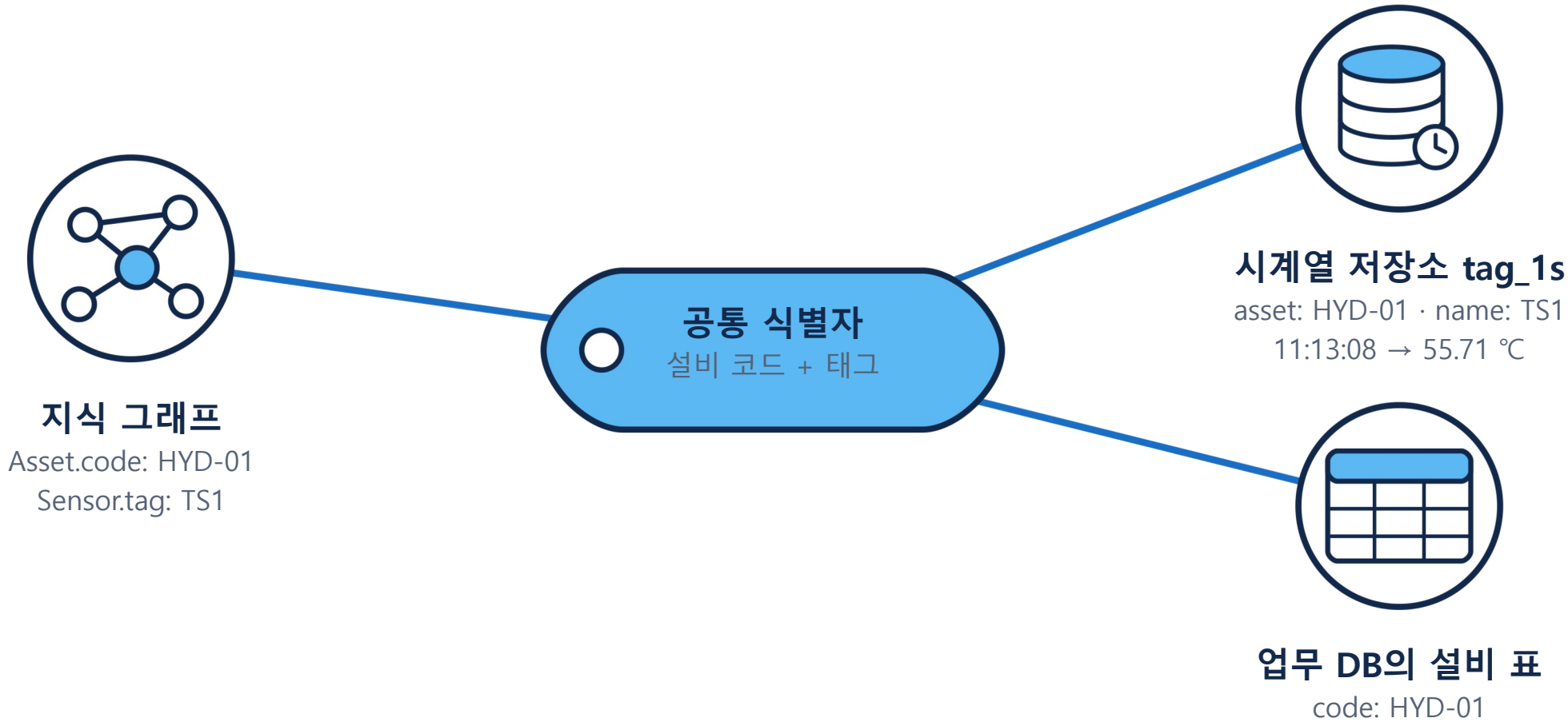
별칭(Alias)

- 같은 대상을 부르는 다른 이름의 목록이다
- comp:cooler: 쿨러, 열교환기
- sv:ts1: 오일 온도, 작동유 온도, TS1

aliases

여러 저장소를 잇는 공통 식별자

저장소마다 같은 설비 코드와 태그 값을 써야, 지식 그래프에서 찾은 대상으로 다른 저장소의 데이터까지 찾아간다.



05

속성과 연결의 정의 · 제약

이론

속성값과 연결이 지킬 조건을 정하고, 어긋난 데이터를 대조한다

클래스 이름 · 의미 · 속성 정의

클래스 정의는 이름과 의미로 범위를 정하고, 속성마다 속성명 · 자료형 · 필수 여부를 적는다.

```
{
  "name": "Sensor",
  "label_ko": "센서",
  "description": "계측 채널. 태그 이름이 실시간 데이터와 연결된다.",
  "properties": [
    {"name": "id",      "type": "string", "required": true},
    {"name": "name",    "type": "string", "required": true},
    {"name": "tag",     "type": "string", "required": true},
    {"name": "unit",    "type": "string"},
    {"name": "virtual", "type": "boolean"}
  ]
}
```

name · label_ko

클래스를 가리키는 하나뿐인 영문 이름과, 화면과 문서에 쓰는 한글 이름이다.

description

클래스의 의미이다. 「계측 채널」이라고 적어 냉각 효율을 계산해 내놓는 채널도 이 클래스에 든다.

속성명 · 자료형 · 필수 여부

properties의 한 줄이 속성 하나이다. tag는 이름이 tag, 자료형은 글자이고, 반드시 있어야 한다.

발췌

원래 정의의 계층 · 준용 표준 칸과 virtual의 설명은 뺐다.

자료형과 허용값

자료형(Data Type)은 칸에 들어갈 값의 종류를 정해, 그 값으로 할 수 있는 비교와 계산의 기준이 된다.

문자열(string)	글자이다. name:「유온 센서」, tag: 'TS1', unit: '℃'
수(number)	크기를 비교하고 계산할 수 있다. normal: 48, limit: 65 「130 bar」처럼 단위를 섞은 글자는 수로 바로 비교할 수 없다.
참 · 거짓(boolean)	둘 가운데 하나이다. virtual: false, avl: true
목록(string[])	값이 여럿일 때 쓴다. aliases: 오일 온도, 작동유 온도, TS1
열거형(enum)	정해 둔 허용값 가운데 하나만 받는다. kind: manipulated(조작) · controlled(결과) · disturbance(교란) manipulated: 팬 속도 · 펌프 부하 · 운전 펌프 선택

필수 속성과 선택 속성

필수 속성이 빠진 인스턴스는 정의에 어긋나지만, 선택 속성은 비어 있어도 정의에 맞는다.

상태 변수	필수: unit · kind	선택: normal · limit
유온 sv:ts1	℃ · controlled	**48 · 65**
토출 압력 sv:ps1	bar · controlled	**182 · 130(낮은 쪽 한계)**
팬 진동 sv:vs1	mm/s · controlled	**0.6 · 2.0**
냉각 효율 sv:ce	% · controlled	**84 · 없음(경보의 CE < 70 %와 구별)**
유량 sv:fs1	l/min · controlled	**9 · 없음**
펌프 부하 sv:load	% · manipulated	**90 · 없음**

관계 정의와 유효한 연결

유효한 연결은 스키마의 조건을 따르며, 이름·방향·양 끝 클래스부터 확인한다.

```
{
  "type": "OBSERVES",
  "from": ["Sensor"],
  "to": ["StateVariable"],
  "cardinality": "N:1",
  "description": "센서가 읽는 상태 변수"
}
(:Sensor)-[:OBSERVES]->(:StateVariable)    유효
(:Sensor)-[:OBSERVES]->(:Asset)            끝 클래스가 다르다
(:StateVariable)-[:OBSERVES]->(:Sensor)    방향이 반대이다
(:Sensor)-[:WATCHES]->(:StateVariable)     정의에 없는 이름

가정: 유온 센서 → 팬 진동을 OBSERVES로 잇는다
양 끝 클래스는 맞지만, 실제 계측 대상은 틀리다
```

type

관계 유형의 이름이다.

from · to

시작 클래스와 끝 클래스이다. 화살표는 from에서 to로 향한다.

cardinality

N:1. 여러 센서가 한 상태 변수를 읽을 수 있다.

유효한 연결(Valid Connection)

연결 조건을 만족한다. 클래스가 맞아도 실제 대상이 맞는지는 별도로 확인한다.

스키마의 제약과 데이터 오류

제약(Constraint)은 데이터가 지킬 조건을 정한 것이어서, 어긋난 데이터를 찾아낼 기준이 된다.

필수 속성

- 센서에는 tag가 있다
- 오류: tag 없는 센서

빠진 값

자료형 · 허용값

- limit은 수, kind는 세 값 중 하나
- 오류: 「130 bar」, kind 'x'

값의 형식

고유한 식별자

- 같은 id는 하나뿐이다
- 오류: sen:ts1 노드가 둘

중복

연결 조건

- OBSERVES: 센서 → 상태 변수
- 오류: 역방향 · 미정의 HAS_PART

양 끝과 이름

06

설비 모델의 설계와 확장

이론

답할 질문에서 대상 · 값 · 연결을 골라 설비 모델을 만든다

질문에서 클래스 · 속성 · 관계로

설계는 답할 질문에서 출발해, 질문에 쓰이는 대상과 값과 연결만 클래스 · 속성 · 관계로 정한다.



질문을 정한다

질문 1: HYD-01의 구성 요소를 재는 센서

질문 2: 그 센서가 읽는 물리량의 한계

질문 3: 오일 쿨러에 들어가는 교체 부품

클래스와 인스턴스를 나눈다

Asset · Component · Sensor

StateVariable · Part

예: HYD-01, 오일 쿨러, 유온 센서, 유온, 쿨러 코어

속성을 정한다

속성명 · 자료형

필수 여부 · 식별자

한계는 수로 둔다

관계를 정한다

이름 · 방향

양 끝 클래스

카디널리티

설비 모델의 속성과 관계 정의

질문에 쓰이는 값은 속성으로, 질문이 건너가는 길은 관계로 적은 것이 설비 모델의 핵심 정의이다.

```
Asset         { id!  name!  code! }
Component     { id!  name!  aliases }
Sensor        { id!  name!  tag!  unit }
StateVariable { id!  name!  unit!  kind!  limit }
Part          { id!  name!  partNo }

HAS_COMPONENT   Asset     -> Component      1:N
MONITORED_BY    Component -> Sensor         N:M
OBSERVES        Sensor    -> StateVariable  N:1
USES_PART       Component -> Part           N:M
```

느낌표

필수 속성이다. 이 다섯 클래스에서 id와 name은 필수이다.

필수로 둔 값

설비의 code, 센서의 tag, 상태 변수의 unit · kind는 없으면 쓸 수 없다.

핵심 정의 발췌

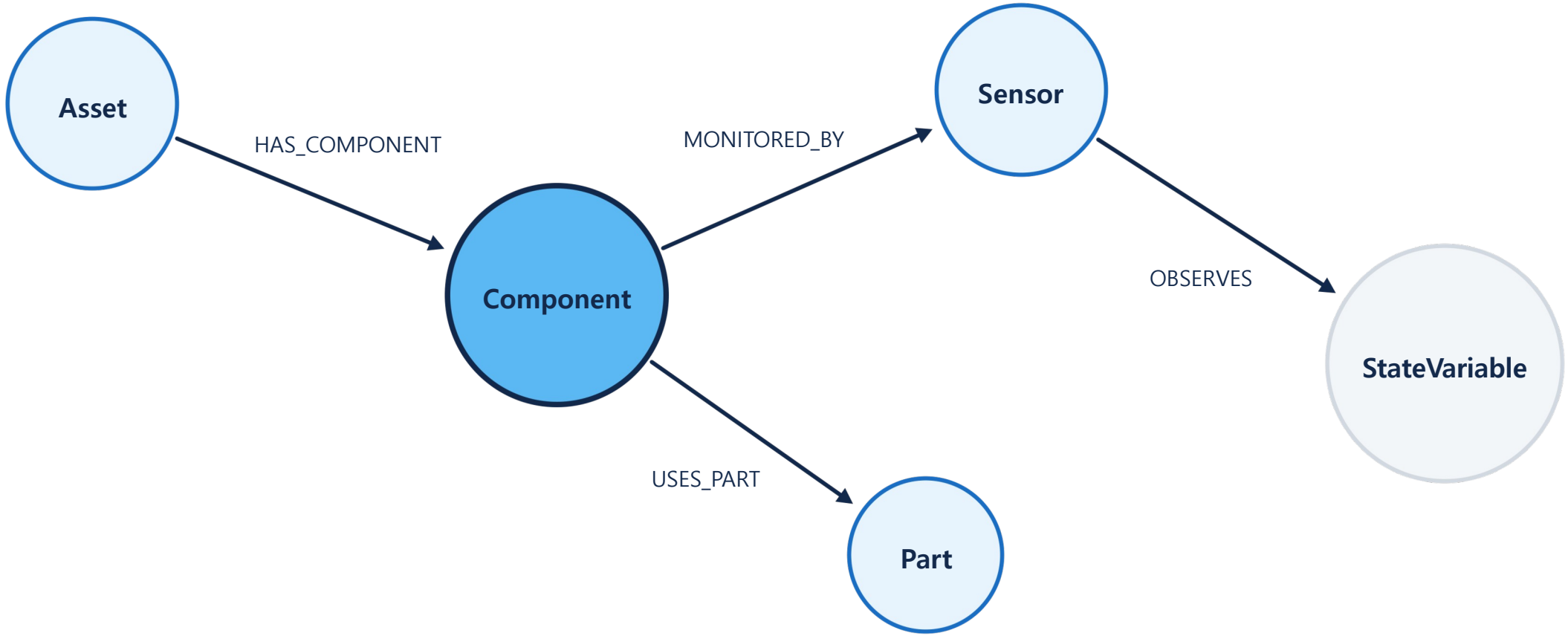
질문에 쓰이는 속성만 골랐다. 전체 정의에는 normal · virtual 등이 더 있다.

HAS_COMPONENT

1:N으로 선언되어 있지만 구성 요소 노드는 세 설비가 함께 가리킨다.

설비 모델의 관계 구조

다섯 클래스를 네 관계로 이으면 세 질문이 모두 구성 요소를 거쳐 가는 구조가 드러난다.



인스턴스 추가와 스키마 변경

대상이 늘면 인스턴스를 더하고, 질문이나 기존 조건이 바뀌면 스키마와 데이터를 함께 살핀다.

인스턴스를 더한다

- 예시 인스턴스를 넣고 처음 질문에 답이 나오는지 본다
- HYD-01 → 오일 쿨러 → 유온 센서 TS1 → 유온 limit 65
- 오일 쿨러 → 쿨러 코어: 질문 3의 답
- 같은 기종 설비가 늘어도 스키마는 그대로이다

자주 일어난다

vs

스키마를 고친다

- 새 질문: 쿨러 코어의 공급사(다섯 클래스 모델에는 없다)
- 추가: Supplier · SUPPLIED_BY(Part → Supplier)
- 답: A정밀 · B-OEM
- 기존 조건 변경(가정): Sensor.unit 선택 → 필수
- 영향: 기존 센서의 단위 누락도 확인한다

드물게, 신중하게

설비 · 진단 · 판단 · 성과의 지식

같은 방식으로 정의한 클래스가 설비 · 진단 · 판단 · 성과의 네 묶음으로 온톨로지에 들어 있다.



설비

Asset, Component, Sensor, Actuator, StateVariable, Part, Supplier

무엇으로 이루어졌는가



진단

AnomalyPattern, Symptom, FailureMode, Cause, Evidence

왜 잘못되었는가



판단

Skill, Decision, DecisionTable, Rule 등 조치 방법과 판단 규칙의 클래스

무엇을 할 수 있는가



성과

Perspective, Objective, Measure 등 목표와 성과 지표의 클래스

무엇이 좋아지는가

07

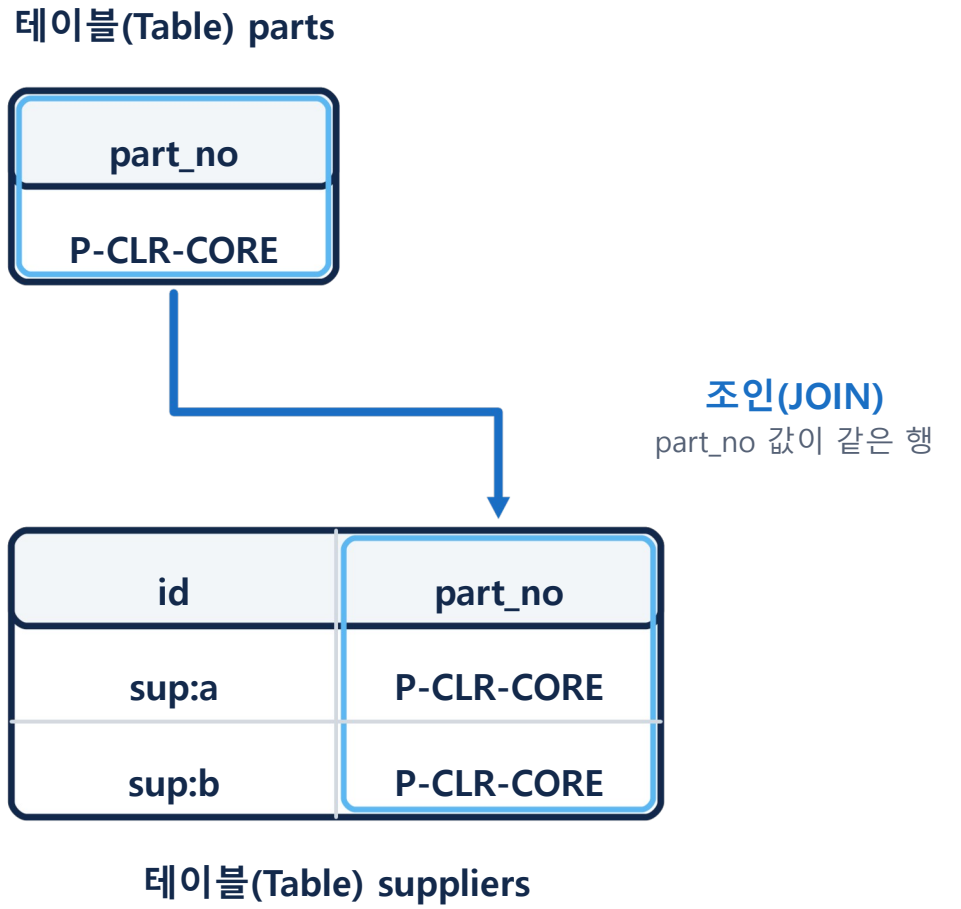
그래프 DB의 저장 구조와 Cypher 쿼리 조회

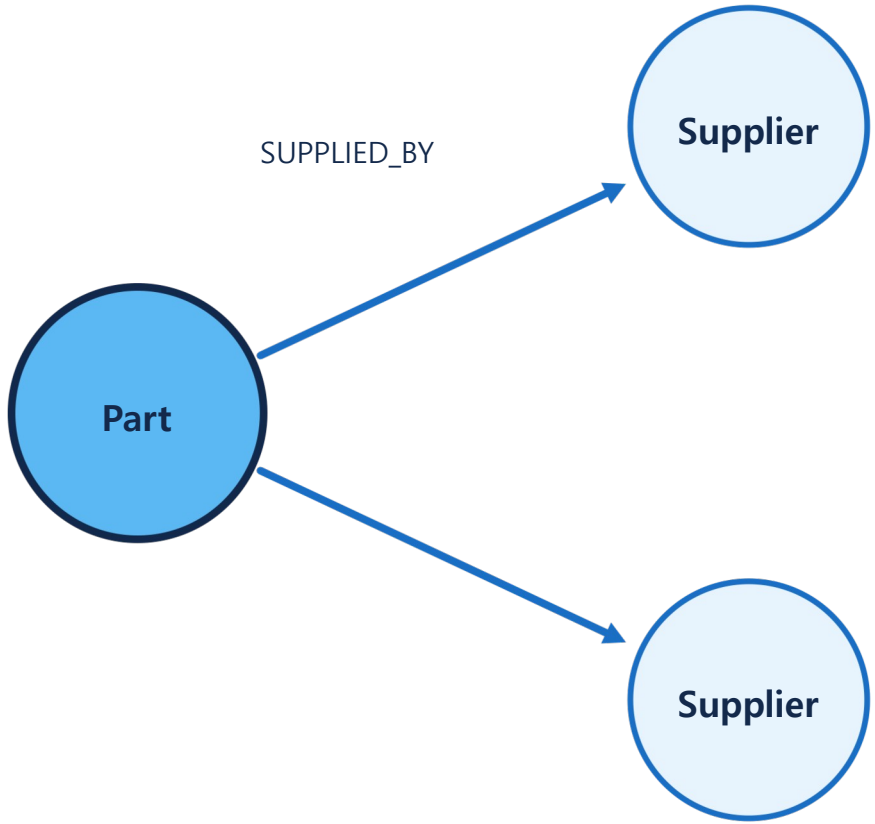
이론

노드 · 레이블 · 관계로 저장되는 모양을 보고, 그 모양을 패턴으로 적어 찾는다

테이블의 조인과 그래프의 연결

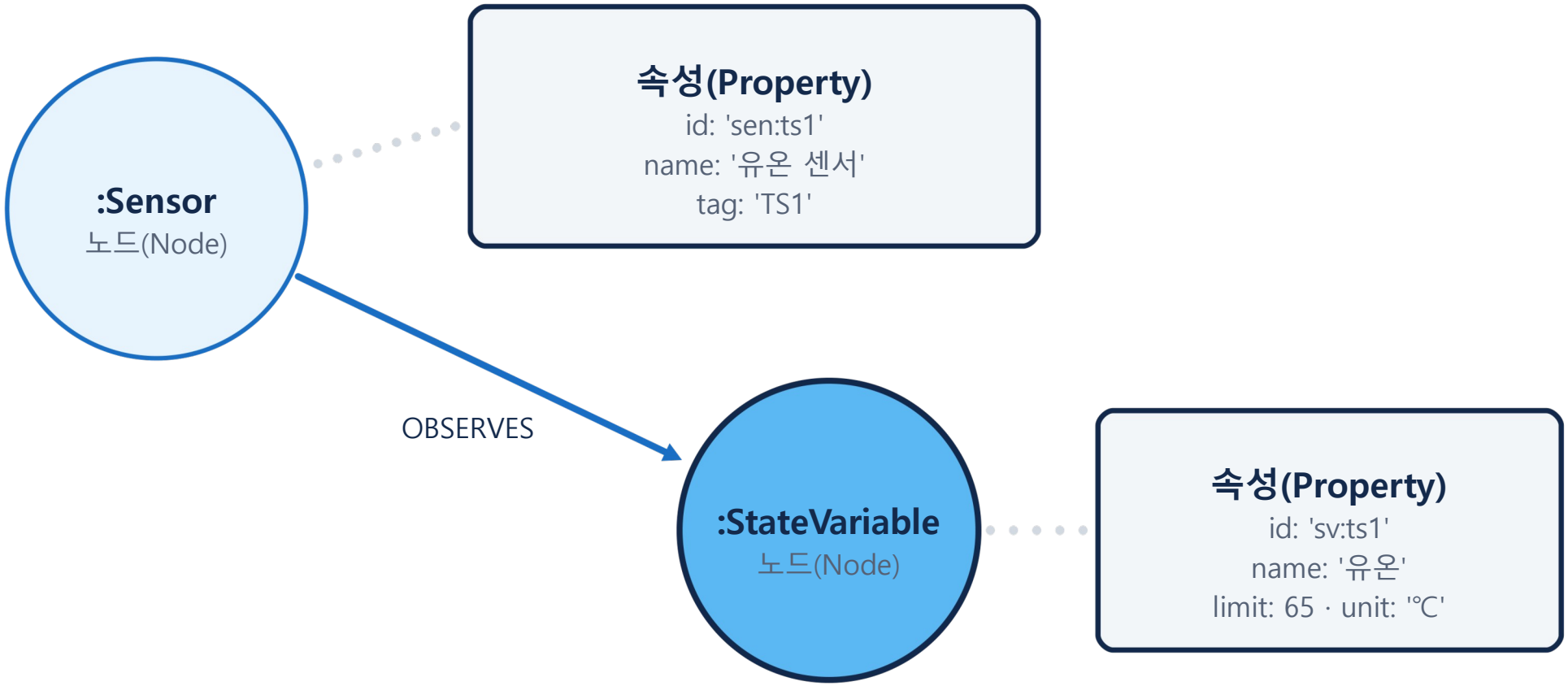
관계형 DB는 테이블의 행을 열 값으로 **조인**하고, 그래프 DB는 저장된 관계를 따라간다.





노드 · 레이블 · 관계 · 속성

그래프 DB는 인스턴스를 **노드**로, 클래스 이름을 **레이블**(Label)로, 값을 **속성**으로 저장한다.



온톨로지 스키마와 DB 제약

스키마는 옳은 데이터의 조건을 정하고, **DB 제약**은 그중 저장할 때 데이터베이스가 직접 막는 조건이다.

온톨로지 스키마

- 클래스 → 레이블, 관계 유형 → 관계 유형, 속성 → 속성으로 옮겨진다
- 필수 속성 · 자료형 · 허용값 · 끝 클래스 · 카디널리티를 정한다
- 별도 검사기가 필수값 · 허용값 · 끝 클래스를 확인한다
- 자료형 전체와 카디널리티는 자동 검사되지 않는다

의미 모델의 정의

vs

DB 제약(DB Constraint)

- 데이터베이스에 걸어 두어 저장하는 순간 검사하는 조건이다
- 유일 제약(Unique Constraint): 같은 레이블의 노드끼리 id가 겹치지 않는다
- 예: id가 sen:ts1인 Sensor 노드를 또 만들면 저장을 거절한다

저장할 때 강제

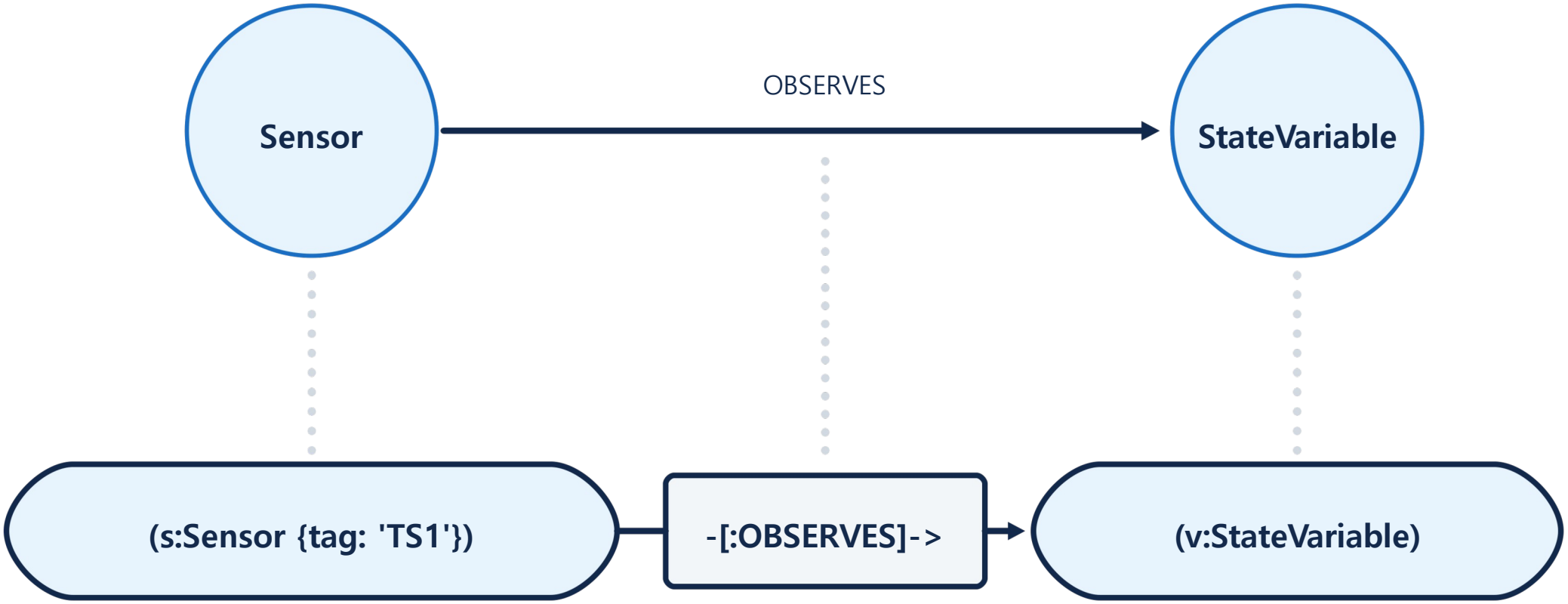
Cypher 쿼리와 실행 화면의 역할

저장된 그래프는 **Cypher 쿼리**로 꺼내고 만들며, 쿼리는 찾을 노드와 관계의 모양을 글자로 적는다.

질의(Query)	데이터베이스에 보내는 요청 한 건이다. 무엇을 찾거나 만들지를 적는다. 쿨러 사건에서 경보로부터 원인 후보를 찾은 것도 그래프에 보낸 질의였다.
Cypher 쿼리(Cypher Query)	Cypher로 적은 질의이다. Cypher는 요청을 노드와 관계의 패턴(Pattern)으로 적는 그래프 DB 질의 언어이다. 예: (c:Component)-[:MONITORED_BY]->(s:Sensor)
질의 실행 화면 사람이 쓰는 창구	사람이 질의를 입력해 실행하고 결과를 표와 그래프로 보는 화면이다. Neo4j Browser 같은 것이 있으며, 포털의 지식 지도와는 별개이다.

노드 · 관계 · 변수 · 속성 조건의 표기

변수(Variable)는 찾은 대상을 다시 부르는 이름이며, 속성 조건으로 검색 대상을 좁힌다.



MATCH · RETURN: 패턴과 결과의 행

MATCH는 패턴에 맞는 연결을 찾고, RETURN은 그 결과에서 돌려받을 항목을 정한다.

```
MATCH (c:Component {name: '오일 쿨러'})
      -[:MONITORED_BY]->(s:Sensor)
RETURN c.name, s.tag


—— 결과 ——
c.name          s.tag
"오일 쿨러"     "TS1"
"오일 쿨러"     "CE"
```

MATCH

이름이 오일 쿨러인 구성 요소에서 계측 관계로 이어진 센서를 찾는다.

RETURN

구성 요소 이름과 센서 태그를 결과의 두 열로 받는다.

행의 수

쿨러 하나에 계측 채널 둘이 연결되어 결과는 두 행이다.

행과 노드

두 행의 오일 쿨러는 같은 노드이다. 이름의 반복이 노드의 중복을 뜻하지 않는다.

RETURN: 속성값과 노드 · 관계

같은 패턴을 찾더라도 속성값을 받으면 표로 읽고, 노드와 관계를 받으면 연결을 그릴 수 있다.

```
MATCH (c:Component {name: '오일 쿨러'})
      -[r:MONITORED_BY]->(s:Sensor)
RETURN c.name, s.tag

── 같은 MATCH, 다른 RETURN ──
RETURN c, r, s

── 반환하는 대상(속성 일부 생략) ──
(:Component)  [:MONITORED_BY]  (:Sensor {tag: 'TS1'})
(:Component)  [:MONITORED_BY]  (:Sensor {tag: 'CE'})
```

찾는 패턴

두 쿼리 모두 같은 쿨러와 센서의 연결을 찾는다.

속성값 반환

c.name, s.tag는 각 노드의 값이다. 이름과 태그 두 열을 받는다.

노드 · 관계 반환

c, r, s는 구성 요소 노드, 계측 관계, 센서 노드 자체이다.

표와 그래프

두 결과 모두 두 행이다. 아래 결과를 그래프로 그리면 쿨러 하나와 센서 둘이 나타난다.

DISTINCT: 반환한 값의 중복 제거

DISTINCT는 반환하는 값의 조합이 같은 행을 하나로 남기며, 저장된 노드를 지우지 않는다.

같은 MATCH의 반환 절	결과	남는 행
RETURN c.name	오일 쿨러 오일 쿨러	2행
RETURN DISTINCT c.name	오일 쿨러	1행
RETURN DISTINCT c.name, s.tag	오일 쿨러 · CE 오일 쿨러 · TS1	2행
중복 제거(DISTINCT)	쿨러 → 계측 채널 둘의 같은 패턴에서 반환 절만 바꾼다 반환한 모든 열의 값이 같을 때 한 행으로 남긴다	결과 정리

COUNT: 결과 행과 서로 다른 대상의 수

COUNT로 행을 셀지 서로 다른 대상을 셀지 정해야 질문에 맞는 개수를 얻는다.

```
MATCH (c:Component {name: '오일 쿨러'})
      -[:MONITORED_BY]->(s:Sensor)
RETURN count(*) AS rows,
       count(DISTINCT c) AS components,
       count(DISTINCT s) AS channels

─── 결과 ───
rows    components    channels
2       1             2
```

집계 함수(COUNT)

여러 행을 개수 하나로 요약한다. count(*)는 입력 행을 센다.

COUNT와 DISTINCT

count(DISTINCT c)는 서로 다른 구성 요소 노드를 센다. 같은 쿨러가 두 번 나와도 하나이다.

세는 대상

count(DISTINCT s)는 서로 다른 센서 노드를 센다. 이름이 아니라 노드 자체를 구별한다.

별명(AS)

계산 결과의 열에 rows · components · channels라는 이름을 붙인다.

Cypher 쿼리: 여러 관계를 잇는 경로 조회

노드와 관계를 번갈아 이어 적으면 설비에서 물리량의 한계까지 질의 한 번으로 따라간다.

```
MATCH (a:Asset {code: 'HYD-01'})
      -[:HAS_COMPONENT]->(c:Component)
      -[:MONITORED_BY]->(s:Sensor)
      -[:OBSERVES]->(v:StateVariable)
RETURN c.name, s.tag, v.name, v.unit, v.limit
ORDER BY s.tag


─── 결과 ───
"오일 쿨러"    "CE"   "냉각 효율"  "%"      NULL
"주 펌프 (A)"  "FS1"  "유량"       "l/min"  NULL
"주 펌프 (A)"  "PS1"  "토출 압력"  "bar"    130
"오일 쿨러"    "TS1"  "유온"       "℃"      65
"쿨러 팬"      "VS1"  "팬 진동"    "mm/s"   2.0
```

출발

설비 코드가 HYD-01인 Asset에서 시작한다.

이어 적은 관계

노드와 관계를 번갈아 적어 세 관계를 한 번에 건넌다.

NULL

값이 없다는 표시이다. 냉각 효율과 유량에는 한계 속성값을 두지 않았다.

결과

HYD-01의 구성 요소에 달린 센서마다 한 행이다. TS1 행이 질문 1 · 2의 답이다.

WHERE: 조건과 값의 존재 여부

WHERE로 한계가 있는 물리량만 남기면, 값이 없는 상태 변수를 결과에서 구별할 수 있다.

```
MATCH (s:Sensor)-[:OBSERVES]->(v:StateVariable)
WHERE v.limit IS NOT NULL
RETURN s.tag, v.name, v.limit, v.unit
ORDER BY s.tag

─── 결과 ───
"PS1"  "토출 압력"  130   "bar"
"TS1"  "유온"       65    "℃"
"VS1"  "팬 진동"    2.0   "mm/s"

─── 빠진 행 ───
CE 냉각 효율 · FS1 유량: limit 값 없음
```

WHERE

MATCH가 찾은 행 가운데 조건에 맞는 행만 남기는 조건 절이다.

IS NOT NULL

속성에 값이 있는지를 묻는다. 크기를 비교하는 조건이 아니다.

빠진 행

냉각 효율과 유량은 센서와 이어져 있지만 limit 값이 없어 빠진다.

크기 비교

limit > 100처럼 비교할 수도 있으나, 단위가 다른 물리량을 한 기준으로 견주지 않는다.

AND · OR: 조건을 괄호로 묶기

쿨러 또는 팬이라는 선택을 먼저 묶어야 두 장치 모두에 한계 값 조건을 적용한다.

```
MATCH (c:Component)-[:MONITORED_BY]->(s:Sensor)
      -[:OBSERVES]->(v:StateVariable)
WHERE (c.name = '오일 쿨러' OR c.name = '쿨러 팬')
      AND v.limit IS NOT NULL
RETURN s.tag, v.name
ORDER BY s.tag
─── 결과 ───
TS1   유온
VS1   팬 진동
─── 괄호를 빼면 ───
CE    냉각 효율
TS1   유온
VS1   팬 진동
```

OR: 어느 하나

오일 쿨러이거나 쿨러 팬인 구성 요소를 고른다.

AND: 모두

고른 장치의 채널 중 한계가 있는 물리량만 남긴다.

괄호: 조건의 묶음

괄호가 없으면 AND가 OR보다 먼저 묶인다.
쿨러 쪽은 한계 값 조건을 거치지 않는다.

구성 요소별 집계와 ORDER BY 정렬

구성 요소를 구별해 채널 수를 집계하면 원래의 다섯 행을 장치별 세 행으로 비교할 수 있다.

```
MATCH (a:Asset {code: 'HYD-01'})
      -[:HAS_COMPONENT]->(c:Component)
      -[:MONITORED_BY]->(s:Sensor)
RETURN c.id AS componentId, c.name AS component,
       count(DISTINCT s) AS channels
ORDER BY channels DESC, componentId


── 결과(componentId 열은 생략) ──
component      channels
오일 쿨러    2
주 펌프 (A)    2
쿨러 팬     1
```

묶는 기준

집계하지 않은 c.id와 c.name의 조합별로 센서를 센다. 같은 이름의 다른 장치도 id로 구별한다.

정렬(ORDER BY)

DESC는 큰 값부터 놓는 내림차순이다. 수가 같으면 componentId 오름차순으로 놓는다.

원래 행과 대조

쿨러: CE · TS1

주 펌프: FS1 · PS1

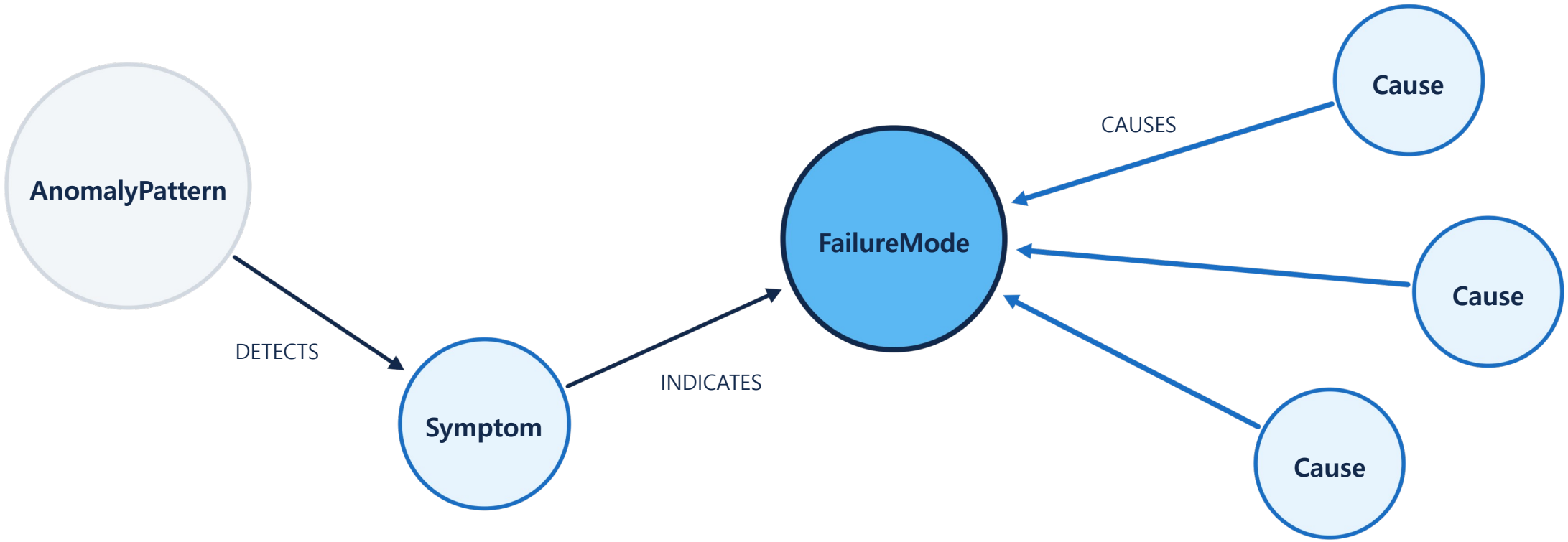
팬: VS1

조회 범위

HYD-01에서 센서로 이어지는 구성 요소를 센다. 센서가 없는 구성 요소는 이 패턴에 나오지 않는다.

관계의 저장 방향과 탐색 방향

관계는 저장된 방향대로 적고 화살표를 거슬러 읽으며, 방향을 뒤집어 적으면 찾지 못한다.



(fm:FailureMode)<-[:CAUSES]-(c:Cause)
역방향 탐색 · 원인 후보

(fm:FailureMode)-[:CAUSES]->(c:Cause)
뒤집어 적은 관계 · 결과 없음

08

Cypher 쿼리: 노드 · 관계 생성과 병합

이론

식별자로 대상을 찾거나 만들고, 관계를 이은 뒤 설계와 대조한다

CREATE · MERGE · SET의 역할

다시 실행해도 같은 대상이 하나로 남게 하려면, 바뀌지 않는 식별자로 찾고 없을 때만 만든다.

```
CREATE (s:Sensor {id: 'sen:ts1',
                  name: '유온 센서',
                  tag: 'TS1'})

MERGE (s:Sensor {id: 'sen:ts1'})
SET s.name = '유온 센서',
    s.tag  = 'TS1',
    s.unit = '℃'

── sen:ts1이 이미 있을 때 CREATE의 결과 ──
Node already exists with label `Sensor`
  and property `id` = 'sen:ts1'
```

CREATE

적은 대로 새 노드를 만든다. 이미 있는지는 보지 않는다. sen:ts1이 이미 있으면 유일 제약에 걸려 거절된다.

MERGE

패턴에 맞는 노드를 찾고 없으면 만든다. 이 예는 id 유일 제약 아래 같은 id로 반복 실행한다.

SET

찾았거나 만든 노드의 속성에 값을 넣는다.

식별자로 찾는다

같은 id를 조건으로 기존 센서를 찾는다. 이름과 태그 같은 값은 SET으로 넣는다.

MERGE의 검색 조건과 SET의 변경값

같은 센서의 이름을 바꿀 때는 식별자로 찾고, 바뀔 이름은 SET으로 넣는다.

```
// 두 쿼리는 같은 초기 상태에서 각각 비교
// 기존 이름: '유온 센서', id: 'sen:ts1'

MERGE (s:Sensor {id: 'sen:ts1'})
SET s.name = '온도 센서'
// 결과: 기존 노드의 이름 변경

MERGE (s:Sensor {id: 'sen:ts1',
                 name: '온도 센서'})
// 결과: id 유일 제약으로 거절
```

실행 전

같은 id의 센서가 있으며 이름은「유온 센서」이다. Sensor.id 유일 제약이 걸려 있다.

식별자로 찾기

id가 같은 노드를 찾고 SET으로 이름만 바꾼다. 센서 노드는 하나 그대로이다.

이름까지 검색 조건에 넣기

id와 새 이름이 모두 맞는 노드가 없다. 만들려다 id 유일 제약으로 거절된다.

반복 실행

첫 쿼리를 다시 실행해도 같은 노드를 찾는다. 대상의 동일성과 바뀌는 속성을 분리한다.

기존 노드를 찾아 관계 생성

관계는 양 끝 노드를 식별자로 먼저 찾은 뒤, 스키마의 방향대로 그 사이에 만든다.

```
MATCH (c:Component {id: 'comp:cooler'}),
      (s:Sensor {id: 'sen:ts1'}),
      (v:StateVariable {id: 'sv:ts1'})
MERGE (c)-[:MONITORED_BY]->(s)
MERGE (s)-[:OBSERVES]->(v)

—— 없는 id로 찾으면 ——
MATCH (c:Component {id: 'comp:cooler'}),
      (s:Sensor {id: 'sen:zz9'})
MERGE (c)-[:MONITORED_BY]->(s)
결과: 만들어진 관계 없음
```

MATCH

이을 노드 셋을 식별자로 찾아 변수 c, s, v에 담는다.

MERGE

변수로 부른 두 노드 사이에 그 관계가 없을 때만 만든다.

방향

화살표는 스키마의 시작 클래스에서 끝 클래스로 향하게 적는다.

끝점 누락

찾는 노드가 하나라도 없으면 아무 관계도 만들어지지 않는다.

정의 · 적재 · 조회 · 검사의 연결

저장이 성공한 것과 설계가 옳은 것은 다르므로, 넣은 뒤 조회와 검사로 정의와 대조한다.



실습: 그래프의 관계 조회

실습

조회 결과와 집계의 대조

관계를 따라 찾은 원래 행과 구성 요소별 채널 수를 대조해 질의가 질문에 맞는지 확인한다.

입력과 질문

- 예제 그래프의 관계 · 방향을 읽는다
- 설비의 구성 요소와 계측 채널을 찾는다

조회 대상

기본 조회

- 한 단계 · 여러 단계 패턴과 조건을 쓴다
- 속성값 표와 연결 그래프를 비교한다

기본 결과

대상별 집계

- 구성 요소별 서로 다른 채널 수를 구해 정렬한다
- 원래 행을 직접 묶어 계산한 수와 대조한다

집계 질의 · 대조표

성공 기준

- 질문에 맞는 대상 · 반환 형태를 얻는다
- 원래 행과 집계 수가 일치한다
- 질의 · 결과 · 조회한 그래프를 기록한다

확인

실습: 기준 지식의 연결 탐색

실습

기준 지식에서 확인할 구조

지식 지도에서 설비에 이어진 대상을 따라가며, 클래스 · 속성 · 관계가 실제로 쓰이는 모습을 확인한다.



목표 · 입력

- 지식 지도에서 설비 하나를 골라 이어진 구성 요소와 센서로 따라간다
- 입력: HYD의 기준 지식

설비에서 출발



범위

- 항목의 클래스와 속성을 확인한다
- 나가는 관계와 들어오는 관계를 구분한다
- 직접 만드는 그래프는 기준 지식과 다른 환경에 둔다

클래스 · 속성 · 방향



산출물 · 성공 기준

- 확인한 클래스 · 속성 · 관계를 적은 탐색 기록
- 출발 대상과 도착 대상이 질문에 맞는다

탐색 기록

실습: 온톨로지 설계

실습

스키마 · 인스턴스의 결과물

설비 자료에서 답할 질문을 정해 스키마 파일과 예시 인스턴스 데이터를 만들고, 자료의 뜻과 대조한다.

스키마 파일

- 클래스 · 속성 · 자료형 · 필수값 · 식별자
- 관계의 유형 · 방향 · 연결 제약

정의

예시 인스턴스 데이터

- 자료에 나오는 대상을 인스턴스로 적고
- 정의한 관계로 잇는다

실제 대상

성공 기준

- 고유 id · 필수값 · 관계 방향이 정의에 맞는다
- 함께 쓰는 부품과 한 구성 요소의 여러 센서가 빠짐없이 이어진다

원래 자료와 대조

실습: 그래프 구현과 검증

실습

저장된 그래프와 의미 검증

저장된 관계가 스키마에 맞아도 실제 대상이 다를 수 있으므로 원자료와 조회 결과를 함께 대조한다.



구현 결과

- 설계한 스키마와 인스턴스를 Cypher로 옮긴다
- 생성 질의 · 저장된 그래프를 남긴다

설계에서 저장으로



연결 검토 사례

- 원자료: 유온 센서 → 유온
- 오류 사본: 유온 센서 → 팬 진동
- Sensor → StateVariable 조건은 둘 다 맞는다

의미가 틀린 연결



성공 기준

- 잘못 고른 끝점을 찾아 수정한다
- 다시 조회한 대상이 원자료와 일치한다
- 중복 · 끝점 검사와 수정 기록을 남긴다

완성본 실행 · 대조

설계에서 조회까지: 같은 정보를 이어 읽기

설계에서 정한 대상과 관계는 그래프에 저장되고, 쿼리는 그 구조에서 질문의 답을 꺼낸다.

정보	설계에서의 자리	저장 · 조회에서의 자리
유온 센서의 이름 · 태그	Sensor 인스턴스의 속성값	**노드의 name · tag**
유온 센서 → 유온	Sensor → StateVariable OBSERVES	**노드 사이의 방향 있는 관계**
유온의 한계	StateVariable의 limit 속성	**연결된 노드에서 읽는 값**
쿨러를 계측하는 센서	Component → Sensor MONITORED_BY	**MATCH로 연결을 찾고 RETURN으로 받는 답**
쿨러 조회 결과 두 행	쿨러 하나 · 서로 다른 계측 채널 둘	**count(*) = 2 count(DISTINCT c) = 1**

오늘 다룬 핵심 개념과 주요 용어 정리

본문에서 정의한 아홉 용어의 정식 명칭과 뜻을 한곳에 모은다.

용어	정식 명칭 · 풀이
클래스	Class 공통 특성을 가진 대상의 유형
인스턴스	Instance 클래스에 속하는 실제 개별 대상
속성	Property 대상에 값을 담는 항목(채운 내용은 속성값)
카디널리티	Cardinality 한 관계에서 양쪽 대상이 몇 개씩 이어질 수 있는지(1:1 · 1:N · N:M)
스키마	Schema 클래스 · 속성 · 관계 · 제약을 일관되게 정한 지식 구조의 정의
식별자	Identifier 같은 대상을 하나로 가리키는, 바뀌지 않는 고유한 값
레이블	Label 그래프 DB에서 노드의 종류를 나타내는 이름
Cypher 쿼리	Cypher Query 노드와 관계의 패턴으로 그래프 DB를 조회하고 만드는 질의 문장
DB 제약	DB Constraint 데이터베이스가 저장할 때 강제하는 규칙

Review Quiz: 핵심 흐름 되짚어보기

배운 내용을 바탕으로 정답을 맞혀 보세요.

Quiz 1. 스키마와 인스턴스

공장에 같은 기종의 유압 파워팩 한 대가 새로 들어와 설비 코드 HYD-04를 받았다.

이 설비를 지식 그래프에 반영하는 방법으로 가장 알맞은 것은?

1. 스키마에 「HYD-04」라는 클래스를 새로 정의한다
2. Asset 클래스에 「넷째 설비」라는 속성을 더한다
3. Asset의 인스턴스 하나를 더하고 구성 요소와 HAS_COMPONENT로 잇는다
4. 기존 asset:hyd-01의 code 값을 HYD-04로 고친다

Quiz 2. 관계의 속성

부품 「펌프 축 씰 키트」를 세 공급사가 공급하며, 단가는 35 · 55 · 20만원, 납기는 2 · 5 · 1일이다.

단가와 납기를 두는 자리로 가장 알맞은 것은?

1. 부품과 공급사를 잇는 SUPPLIED_BY 관계의 속성
2. Part 클래스의 속성
3. Supplier 클래스의 속성
4. 「단가」라는 새 클래스의 인스턴스

Quiz 3. 저장을 막는 조건

Sensor 노드의 id에 유일 제약이 걸린 그래프 DB에 아래 네 가지를 저장하려 한다.

저장하는 순간 데이터베이스가 직접 거절하는 것은?

1. **tag가 빠진 센서 노드**
2. **이미 있는 sen:ts1과 같은 id를 가진 새 Sensor 노드**
3. **센서에서 설비로 그은 OBSERVES 관계**
4. **kind에 허용값이 아닌 'x'를 넣은 상태 변수**

정답과 판단 기준

세 문항은 유형과 대상, 값이 딸린 자리, 제약을 지키게 하는 주체를 구분하는지 확인한다.

문항	판단 기준	정답
Quiz 1	새로 생긴 것은 유형이 아니라 대상 하나이다. 스키마는 그대로 두고 인스턴스와 관계를 더한다.	**3번**
Quiz 2	단가와 납기는 부품과 공급사의 짝마다 달라진다. 짝에 딸린 값은 둘을 잇는 관계의 속성으로 둔다.	**1번**
Quiz 3	센서의 id 유일 제약은 저장을 막는다. 필수값 · 허용값 · 끝 클래스는 별도 검사가 찾는다.	**2번**